

Ejercicio Integrador Jr: Sistema de Comisiones Automáticas - Fintexa

Historia Empresarial

El Contexto de Fintexa

Fintexa es una empresa líder en medios de pago digitales en Latinoamérica, procesando más de 45,000 transacciones diarias a través de su plataforma construida con microservicios en .NET Core 8. Con integraciones críticas a Visa y Coelsa, y cumplimiento estricto de estándares PCI DSS, cada decisión técnica impacta directamente la rentabilidad y confianza del negocio.

La Crisis de las Comisiones Manuales

Marzo 2024 - Oficinas Centrales de Fintexa, Sala de Juntas Ejecutiva

Ana Rodríguez, CFO de Fintexa, observaba preocupada los números en su pantalla. Los reportes financieros del primer trimestre mostraban una realidad alarmante: errores en el cálculo de comisiones estaban costando a la empresa \$2.3 millones de dólares mensuales.

"No podemos seguir así", murmuró mientras revisaba el informe que había preparado el equipo de Análisis Financiero.

Los Protagonistas

Ana Rodríguez - CFO: Responsable de la rentabilidad financiera, presionada por la junta directiva para resolver el problema de las comisiones.

Carlos Mendoza - Analista Financiero Senior: Líder del equipo de 3 analistas que calculan manualmente las comisiones usando Excel. 15 años de experiencia, pero reconoce las limitaciones del proceso actual.

María González - Team Lead de Desarrollo: Responsable del equipo de 6 desarrolladores backend. Especialista en arquitecturas de microservicios y patrones de diseño.

Diego Fernández - Desarrollador Jr: Recién incorporado al equipo, asignado para resolver el problema de automatización de comisiones. Su primera tarea crítica en Fintexa.

El Problema Actual

El proceso de cálculo de comisiones en Fintexa sigue un flujo manual que se ha vuelto insostenible:

- Extracción Manual:** Cada lunes, Carlos y su equipo extraen datos de transacciones de 5 bases de datos diferentes
- Consolidación en Excel:** Utilizan una macro de Excel de 2,500 líneas desarrollada hace 4 años

3. Cálculo Manual: Aplican diferentes fórmulas según el tipo de transacción:

- Tarjetas de crédito: 2.8% + \$0.30 por transacción
- Tarjetas de débito: 1.9% + \$0.25 por transacción
- Transferencias bancarias: 1.2% + \$0.15 por transacción
- Pagos QR: 0.8% + \$0.10 por transacción
- Comercios premium: 20% de descuento en todas las comisiones
- Volumen alto (>\$10,000/mes): 15% de descuento adicional

4. Validación Cruzada: Verifican manualmente una muestra del 10% de los cálculos

5. Reporte Final: Generan reportes para facturación el viernes

Las Métricas del Desastre

Los números no mentían:

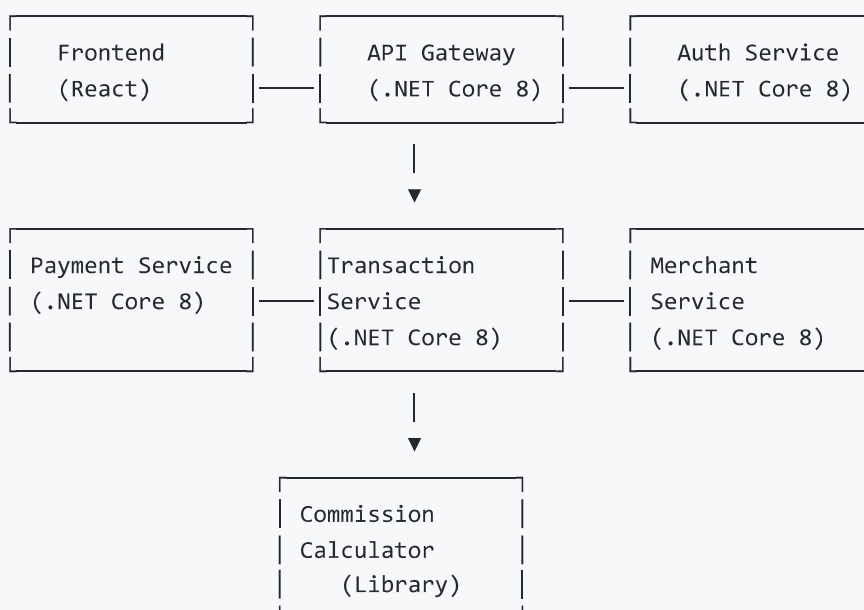
- **12% de tasa de error** en cálculos de comisiones
- **68 horas semanales** del equipo dedicadas solo a cálculos
- **\$2.3M USD mensuales** en pérdidas por errores (cobros incorrectos, disputas, retrabajos)
- **1,200 comercios** han presentado quejas por discrepancias en el último trimestre
- **47 días promedio** para resolver disputas de comisiones

La Presión del Negocio

Durante la junta directiva de marzo, el CEO había sido claro: "O resolvemos esto en 6 semanas, o consideraremos externalizar todo el procesamiento de pagos. No podemos permitirnos seguir perdiendo credibilidad con nuestros comercios."

Documentación Técnica Actual

Arquitectura del Sistema de Pagos



Tipos de Transacciones Actual

```
public enum TransactionType
{
    CreditCard,
    DebitCard,
    BankTransfer,
    QRPayment
}

public enum MerchantTier
{
    Standard,
    Premium,
    HighVolume
}

public class Transaction
{
    public Guid Id { get; set; }
    public decimal amount { get; set; }
    public TransactionType Type { get; set; }
    public Guid MerchantId { get; set; }
    public DateTime ProcessedAt { get; set; }
}
/* ejemplos de Transaction
new Transaction
{
    Id = Guid.NewGuid(),
    amount = 1000,
    Type = TransactionType.CreditCard,
    MerchantId = _merchant.Id,
    ProcessedAt = DateTime.UtcNow
}
*/

public class Merchant
{
    public Guid Id { get; set; }
    public string name { get; set; }
    public MerchantTier Tier { get; set; }
    public decimal MonthlyVolume { get; set; }
}
/* ejemplos de Merchant
new Merchant
{
    Id = Guid.NewGuid(),
    name = "Comercio A",
    Tier = MerchantTier.Standard,
    MonthlyVolume = 5000
}
*/
```

El Problema Técnico

El equipo de desarrollo había intentado crear un calculador básico:

```
public class CommissionCalculator
{
    public decimal CalculateCommission(Transaction transaction, Merchant merchant)
    {
        decimal baseCommission = 0;

        if (transaction.Type == TransactionType.CreditCard)
        {
            baseCommission = transaction.Amount * 0.028m + 0.30m;
        }
        else if (transaction.Type == TransactionType.DebitCard)
        {
            baseCommission = transaction.Amount * 0.019m + 0.25m;
        }
        // ... más if-else

        // Aplicar descuentos
        if (merchant.Tier == MerchantTier.Premium)
        {
            baseCommission *= 0.8m; // 20% descuento
        }

        return baseCommission;
    }
}
```

Problemas identificados:

- Código no extensible (agregar nuevos tipos requiere modificar el método)
- Lógica de negocio dispersa y difícil de mantener
- Testing complejo debido al acoplamiento
- Violación del principio Open/Closed

Conversaciones con Stakeholders

Reunión 1: Análisis del Problema (15 de Marzo, 2024)

Ana (CFO): Carlos, necesito que me expliques exactamente dónde están fallando nuestros cálculos de comisiones.

Carlos (Analista Senior): Ana, el problema principal es la complejidad de las reglas. Tenemos 4 tipos de transacciones, 3 niveles de comercios, descuentos por volumen, promociones especiales... la macro de Excel ya no puede manejar tantas variables.

María (Team Lead): Desde el lado técnico, hemos intentado automatizar, pero cada vez que cambian las reglas de negocio, tenemos que reescribir todo el código.

Ana: ¿Qué tan frecuente cambian las reglas?

Carlos: En promedio, cada 6 semanas. La semana pasada cambiaron las comisiones de QR, hace un mes agregaron un descuento especial para comercios nuevos...

María: Por eso necesitamos una arquitectura que permita agregar nuevas reglas sin tocar el código existente.

Ana: ¿Cuánto tiempo necesitan para una solución automatizada?

María: Si hacemos esto bien, con un patrón de diseño apropiado, 6 semanas. Diego será el desarrollador principal del proyecto.

Reunión 2: Definición de Requerimientos (20 de Marzo, 2024)

Diego (Desarrollador Jr): María, revisé los números que me pasó Carlos. ¿Realmente procesamos 45,000 transacciones diarias?

María: Correcto. Y cada una necesita su comisión calculada en tiempo real, no podemos esperar al proceso batch de los lunes.

Carlos: Las reglas actuales son:

- **Tarjetas de crédito:** 2.8% + \$0.30
- **Tarjetas de débito:** 1.9% + \$0.25
- **Transferencias:** 1.2% + \$0.15
- **QR:** 0.8% + \$0.10
- **Comercios premium:** 20% descuento
- **Alto volumen** (> \$10,000/mes): 15% descuento adicional

Diego: ¿Los descuentos se aplican en secuencia?

Carlos: Exacto. Primero el descuento por tier, después por volumen.

María: Diego, necesito que uses un patrón. Cada tipo de comisión será una estrategia diferente.

Diego: ¿Strategy? ¿o más simple un switch-case?

María: No. El switch-case es exactamente lo que tenemos ahora y no escala. Con Strategy, cada regla es una clase independiente, testeable y extensible.

Reunión 3: Validación de Approach (25 de Marzo, 2024)

Ana: ¿Cómo validaremos que la nueva solución es correcta?

Carlos: Tengo un dataset de 10,000 transacciones del mes pasado con los cálculos manuales. Podemos usarlo como ground truth.

Diego: Perfecto. Implementaré pruebas unitarias para cada strategy y una prueba de integración con ese dataset.

María: Además, necesitamos monitorear el performance. 45,000 transacciones diarias significa aproximadamente 0.5 transacciones por segundo, con picos de hasta 10 TPS.

Ana: ¿Qué pasa si necesitamos agregar una nueva regla la próxima semana?

Diego: Con Strategy Pattern, solo creo una nueva clase que implemente la interfaz ICommissionStrategy. No toco el código existente.

Carlos: ¿Y los reportes? Necesito seguir generando los mismos reportes para los comercios.

Diego: El output será idéntico, solo cambia cómo se calcula internamente.

Fases del Ejercicio

Fase 1: Análisis (15 minutos)

Objetivo: Analizar el problema actual y diseñar la solución usando Strategy Pattern.

Actividades:

1. Identificar las variaciones :

- ¿Qué cambia entre diferentes tipos de transacciones?
- ¿Qué permanece constante?
- ¿Dónde están los puntos de extensión?

2. Definir las abstracciones :

- ¿Cuál será la interfaz común?
- ¿Qué parámetros necesita cada estrategia?
- ¿Cómo se aplicarán los descuentos?

3. Planificar la implementación :

- ¿Cuántas clases de estrategia necesitamos?
- ¿Cómo se configurará el contexto?
- ¿Dónde vivirá la lógica de selección de estrategia?

Entregables:

- Diagrama de clases básico
- Lista de estrategias a implementar
- Identificación de requisitos funcionales y no funcionales

Fase 2: Diseño (15 minutos)

Objetivo: Crear el diseño detallado de interfaces y clases.

Actividades:

1. Diseñar la interfaz ICommissionStrategy :

- Método para calcular comisión base
- Parámetros necesarios
- Tipo de retorno

2. Diseñar el contexto CommissionCalculator :

- Cómo maneja las diferentes estrategias
- Lógica de aplicación de descuentos
- Configuración de estrategias

3. Planificar las estrategias concretas :

- Una estrategia por tipo de transacción

- Implementación específica de cada una
- Validaciones necesarias

Entregables:

- Interfaces definidas
- Estructura de clases
- Flujo de ejecución documentado

Fase 3: Implementación (30 minutos)

Objetivo: Implementar la solución completa usando Strategy Pattern.

Actividades:**1. Implementar la interfaz base (5 min):**

- `ICommissionStrategy`
- Definir el contrato

2. Implementar estrategias concretas (15 min):

- `CreditCardCommissionStrategy`
- `DebitCardCommissionStrategy`
- `BankTransferCommissionStrategy`
- `QRPaymentCommissionStrategy`

3. Implementar el contexto (5 min):

- `CommissionCalculator`
- Lógica de selección de estrategia
- Aplicación de descuentos

Entregables:

- Código funcionando
- Casos de prueba básicos
- Validación con datos de ejemplo

Fase 4: Refactoring y Optimización (15 minutos)

Objetivo: Mejorar el código y prepararlo para producción.

Actividades:**1. Revisar y optimizar :**

- Eliminar código duplicado
- Mejorar Constantes/Variables/métodos
- Agregar validaciones
- Mantenibilidad y Metricas de calidad

2. Agregar extensibilidad (opcional):

- Factory pattern para crear estrategias
- Configuración externa de reglas

- Logging y monitoreo básico

3. Testing y documentación (opcional):

- Pruebas unitarias
- Documentación de uso
- Ejemplos de extensión

Entregables:

- Código refactorizado
- Pruebas unitarias completas
- Documentación técnica

Criterios de Evaluación

Técnico (70%)

- Implementación correcta del Pattern
- Código limpio y mantenible
- Pruebas unitarias completas
- Manejo correcto de descuentos

Funcional (20%)

- Cálculos correctos para todos los tipos
- Aplicación correcta de descuentos
- Performance adecuado

Bonus (10%)

- Uso de Factory Pattern para estrategias
- Configuración externa de reglas
- Logging implementado
- Agregar Estrategia Crypto 0.0255% + \$0.10 y No aplica descuentos extras para Comercios premium
- Llego un cambio para QR: 0.95% + \$0.10 por transacción, pero si monto es menor 2000 solo +\$0.05

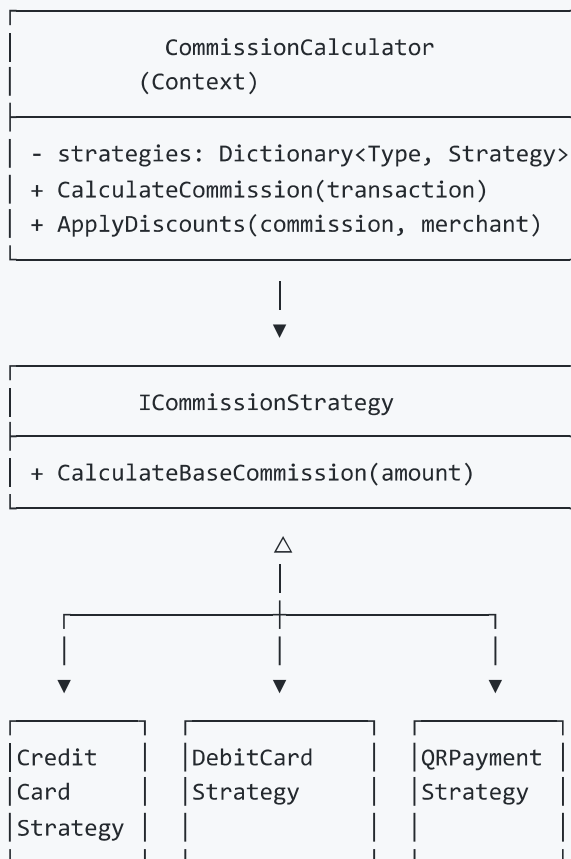
Solución Completa

Diseño Final

El sistema implementado utiliza Strategy Pattern para encapsular cada algoritmo de cálculo de comisiones en una clase separada, permitiendo:

1. **Extensibilidad:** Nuevos tipos de comisión requieren solo una nueva clase
2. **Mantenibilidad:** Cada algoritmo está aislado y es fácil de modificar
3. **Testabilidad:** Cada estrategia se puede probar independientemente
4. **Reutilización:** Las estrategias pueden combinarse y reutilizarse

Arquitectura de la Solución



Impacto del Proyecto

Resultados después de 2 meses de implementación:

- **0.1% tasa de error** (reducción del 99.2%)
- **4 horas semanales** dedicadas a validación (reducción del 94%)
- **\$2.1M USD ahorrados** mensualmente
- **15 segundos** para agregar una nueva regla de comisión
- **100% automatización** del proceso de cálculo

Testimonio de Carlos (Analista Senior): "Lo que antes nos tomaba 68 horas semanales, ahora se ejecuta automáticamente. Podemos enfocarnos en análisis de valor agregado en lugar de cálculos manuales. Diego y María revolucionaron nuestra operación."

Testimonio de Ana (CFO): "Esta implementación no solo resolvió nuestro problema inmediato, sino que nos posicionó para crecer. Ahora podemos lanzar nuevos productos y reglas de comisión en días, no en meses."

El Strategy Pattern demostró ser la solución perfecta para un problema de negocio real, mostrando cómo un patrón de diseño bien aplicado puede generar valor tangible y medible en una organización.